

Dwarf — Garden-Gnome Cat Deterrent — Design

- **Date:** 2026-09-18
- **Status:** Approved design, pre-implementation (revised 2026-09-20: range 5–6 m, head aim)

1. Goal

A garden gnome that watches the backyard, detects cats, turns its head toward them, and sprays a short burst of water at a cat to teach it to stay away. The system must deter without hurting: short bursts, head aim only where the jet has spread out, body aim at close range, no firing at very close range, strict shot limits.

2. Requirements and constraints

Topic	Decision
Coverage	Targets 2–6 m from the gnome
Hours	Daytime only. No night vision. Detection pauses when dark
Who may get wet	Cats are the only targets. People in the yard are never targeted; collateral wetting is acceptable
Aim point	Head beyond 4 m (where the jet arrives as spread spray); body between 2 and 4 m (where the jet is still a hard stream)
Minimum range	No firing within ~2 m of the gnome (drawn as a no-fire zone)
Shot policy	200–400 ms bursts, cooldowns and caps (\$5.5)
Camera + brain	iPhone 6s, iOS 15, jailbroken (checkm8, semi-tethered)
Actuator controller	Any ESP32 with BLE (ESP32, S3, C3). Not ESP32-S2 (no BLE)
Phone-to-ESP32 link	BLE
Water	3 L opaque tank inside the gnome base, 12 V diaphragm pump (~4 bar)
Power	Mains outlet nearby (must be RCD/GFCI protected). Only 12 V enters the gnome
Enclosure	Hollow garden gnome ≥ 50 cm tall, body fixed, head pans, nozzle tilts
Detection	ML from day 1: YOLO (COCO-pretrained, cat class) via CoreML

3. Architecture

```
+----- GNOME -----+
| HEAD:  nozzle (tilt servo)  <- pan servo in neck |
| DRY ZONE (torso) |
|   iPhone 6s  - Swift app "DwarfApp" |
|   camera 1080p -> motion blobs -> crops/tiles -> YOLO "cat" |
|   -> Tracker -> FirePolicy -> Aimer -> BLE command |
|   web UI :8080 over home WiFi (live view, calibration, logs) |
|   ESP32 - actuator + safety watchdog |
|   servos, valve, pump enable, iPhone charger switch, fan, temp |
| ----- sealed divider, drip loops ----- |
| WET ZONE (base) |
|   3 L tank, float switch, 12 V diaphragm pump, NC solenoid valve |
+-----+
12 V cable <- 12 V PSU in outdoor box at RCD-protected outlet
```

The iPhone makes every decision. The ESP32 executes commands and enforces its own hard safety limits, and fails safe when the phone goes silent.

4. Hardware

4.1 Bill of materials

Part	Pick	Notes
Camera/compute	iPhone 6s (owned)	Fixed behind belly/lantern window
Wide lens	Clip-on 0.6x wide lens (~110° FOV)	Covers yard from gnome position
Controller	ESP32 with BLE (owned)	Classic, S3 or C3
Pan servo	DS3218 (preferred) or MG996R, metal gear	In neck, turns head $\pm 60^\circ$
Tilt servo	MG90S metal gear	In head, tilts nozzle
Nozzle	Brass adjustable jet nozzle, 1–1.5 mm orifice	In mouth/pipe
Pump	12 V diaphragm pump, ~4 bar (60 psi), built-in pressure switch	Wet zone, draws from tank via tube. 6 m reach needs far less pressure than 10 m
Valve	12 V normally-closed solenoid valve, 1/4"	Wet zone; short silicone tube with slack loop up neck to nozzle
Tank	3 L opaque container with refill cap	Wet zone, refill via rear hatch. ~30 ml per shot $\approx$ 100 shots
Tank sensor	Float switch	Pump never runs dry
Temp sensor	DS18B20	Dry zone
Fan	5 V fan + vents with insect mesh	Dry zone
Drivers	Logic-level N-MOSFETs + flyback diodes	Pump, valve, fan
Charger switch	High-side P-MOSFET / load switch on iPhone USB 5 V	ESP32-controlled
iPhone charge port	USB-A socket with Apple D+/D- divider (1 A signature) or USB smart-charge module	Otherwise iPhone limits to 500 mA
PSU	12 V 5 A in outdoor-rated box at the outlet	No mains voltage inside gnome
Buck converters	12→6 V 5 A (servos, + 1000 μ F cap), 12→5 V 3 A (ESP32, fan, iPhone)	Separate servo rail avoids ESP32 brownouts
Gnome	Hollow, $\geq$ 50 cm, light-colored, base ~20–25 cm wide	Placed in shade

4.2 Gnome layout

- **Wet zone (base):** tank, pump, valve, float switch. Drain holes in the floor. Water weight at the bottom stabilises the gnome.
- **Dry zone (torso):** iPhone, ESP32, bucks, MOSFET board, DS18B20, fan. Separated from the wet zone by a sealed divider; tubes and cables pass through with drip loops.
- **Camera window:** clear glass/acrylic in the belly or lantern, wide lens pressed close to it (< 2 mm) with a small hood to avoid reflections and glare.
- **Head:** pan servo in the neck carries the head; tilt servo and nozzle inside the head.
- **Heat:** light colour, shade placement, vents high and low, fan controlled by ESP32.

4.3 Power and wiring

- 12 V in → pump (MOSFET), valve (MOSFET), both with flyback diodes.
- 12 V → 6 V buck → both servos (common ground, bulk capacitor).
- 12 V → 5 V buck → ESP32 5 V pin, fan (MOSFET), iPhone USB socket (high-side switch).
- Float switch → GPIO with pull-up. DS18B20 → GPIO with 4.7 k Ω pull-up.

- Pin map per board lives in `firmware/include/pins.h`.

5. iPhone app

Swift, deployment target iOS 15. Code split:

- **DwarfCore** — Swift package with pure logic (no UIKit, no AVFoundation, no CoreML). Runs and is tested on macOS with `swift test`.
- **DwarfApp** — Xcode app containing the iOS adapters that feed **DwarfCore** and execute its decisions.

5.1 Modules

Module	Location	Responsibility
MotionDetector	DwarfCore	Background subtraction on low-res grayscale; returns blobs
Scheduler	DwarfCore	Chooses which crops/tiles go to the detector each cycle
Tracker	DwarfCore	Associates detections into tracks; confirmation; stillness
Aimer	DwarfCore	Calibration fit; pixel ground point → (pan, tilt)
FirePolicy	DwarfCore	Welfare and safety rules; decides aim / shoot / nothing
Protocol	DwarfCore	Encode/decode BLE JSON messages
CameraSource	DwarfApp	AVCaptureSession, 1080p frames at ~10 fps
CatDetector	DwarfApp	CoreML YOLO inference on 640×640 inputs
ActuatorLink	DwarfApp	CoreBluetooth central; commands, heartbeat, status
PowerManager	DwarfApp	Battery hysteresis, thermal state, darkness pause
WebServer	DwarfApp	NWListener HTTP server on port 8080
EventStore	DwarfApp	Event log + dataset capture on disk

5.2 Detection pipeline

1. **CameraSource** delivers 1920×1080 frames, throttled to ~10 fps. At 6 m a cat is about 75 px long in 1080p, which is enough; 4 K would cost roughly four times the pixel work in heat, memory and JPEG time for no gain at this range.
2. **MotionDetector** runs on a 480×270 grayscale downscale: exponential running-average background ($\alpha = 0.05$), absolute difference, threshold 25, dilation, connected components, minimum blob area 20 px. Blobs inside masked zones are dropped.
3. **Scheduler** builds the detector inputs for this cycle:
 - **Motion crops:** for up to 2 blobs, a 640×640 crop from the 1080p frame centred on the blob (clamped to frame). Blobs larger than 640 px get a larger square crop scaled down to 640.
 - **Sweep tile:** one tile per cycle from a 2×1 split of the 1080p frame (960×1080 each, ≥ 128 px overlap), scaled to 640 on the long side and letterboxed; a cat is about 50 px there. Round-robin, so the whole frame is swept every 2 cycles. This catches cats sitting still, which motion detection loses once the background adapts.
4. **CatDetector** runs YOLO11n (fallback YOLOv8n) exported to CoreML with `imgsz=640` and embedded NMS. Only class `cat` is kept. Boxes map back to full-frame coordinates.
5. Cycle budget (crops + tile per cycle, cycle rate) is set from the M0 benchmark.

Model licence note: Ultralytics YOLO is AGPL-3.0; acceptable for this private project.

5.3 Tracker

- Detections are associated to tracks by nearest ground point (bbox bottom-centre), normalised frame coordinates, gate 0.05.
- A track is **confirmed** when 2 of its last 3 detector evaluations returned `cat` with confidence ≥ 0.5 .
- A track is **still** when its ground-point speed over the last 1 s is below 0.02 frame-widths per second.
- A track is dropped after 3 s without a detection.

5.4 Aimer

Ground solution.

- Input: calibration points $(x, y) \rightarrow (\text{pan}, \text{tilt}, \text{range_m})$, where (x, y) is a normalised image point where water landed on the ground and `range_m` is the distance typed in during calibration.
- Model: per axis, 2D quadratic least squares: $v = a_0 + a_1 \cdot x + a_2 \cdot y + a_3 \cdot x^2 + a_4 \cdot x \cdot y + a_5 \cdot y^2$. Requires ≥ 8 points.
- A third fit of the same shape gives `range_m` for any image point.
- Output: per-point residuals (degrees, metres) for the UI.

Aim point by range, using the range of the track's ground point:

Range	Aim	Image point used
< 2 m	no fire (§5.5)	—
2–4 m	body	ground point (bbox bottom-centre)
> 4 m	head	bbox top-centre for pan; ground solution + height offset for tilt

The jet is a hard, coherent stream up close and arrives as spread-out spray further away, so head aim is only allowed where it lands soft. At 6 m the $\sim 1^\circ$ of servo play is about 10 cm, so a head shot lands on the head, neck or shoulders.

Height offset. $\text{tilt_head} = \text{tilt_ground} + \Delta(\text{range})$, where Δ comes from a small calibrated table (§8) of the degrees needed to raise the impact point by 25 cm, linearly interpolated by range and clamped to the calibrated span.

Guards. Results outside servo limits, or from image points outside the calibrated area, are flagged; `FirePolicy` refuses to shoot a flagged solution.

5.5 FirePolicy

Inputs: mode, tracks, masks, clock, ESP32 status. Outputs per cycle: `aim`, `shoot`, `park`, or nothing.

- **Head following:** while any confirmed track exists, send `aim` toward the highest-confidence track (throttled to 5 Hz). With no confirmed track for 10 s, send `park`.
- **Shoot** only when all hold:
 - mode is `live` (or `calibration` for manual test shots);
 - track confirmed and still;
 - ground point outside the no-fire zone and other masks;
 - within the active window (default 07:00–20:00) and scene not too dark;
 - ≥ 10 s since the last shot at this track, ≤ 3 shots per track, ≤ 20 shots per hour;
 - ESP32 status: armed, tank ok, no fault;
 - the `Aimer` solution is not flagged.
- Aim point follows the range table in §5.4: body at 2–4 m, head beyond 4 m.
- Burst length: default 300 ms, configurable 200–400 ms (calibration shots 150 ms).

5.6 Modes

Mode	Detection	Head follows	Pump	Shots
disarmed	off	no	off	no
dry-run (default after install)	on	yes	off	logged as "would fire"
live	on	yes	on	yes
calibration	off	manual jog	on	manual 150 ms test shots

Mode persists across app restarts.

5.7 PowerManager

- **Battery:** charger ON at $\leq 40\%$, OFF at $\geq 80\%$ (`UIDevice.batteryLevel`), sent as `charge` commands.
- **Thermal (`ProcessInfo.thermalState`):** `serious` → halve cycle rate, fan ON; `critical` → pause detection, disarm, fan ON. Resume on `fair` or better.
- **Darkness:** mean luma (0–255) of the 480×270 frame below 40 for 60 s → pause detection until it stays above 40 for 60 s.

5.8 Web UI (port 8080, home WiFi)

- HTTP Basic auth, password set on first launch.
- Pages: live snapshot (auto-refresh), mode switch, status (link, battery, temperature, tank, faults, shots today), calibration (§8), mask editor (polygons: ignore zones and the no-fire zone), event log with images, settings (all thresholds in this spec), export/import of calibration + masks + settings as JSON, dataset export.

5.9 EventStore and dataset capture

- Every confirmed track, every shot and every "would fire" decision creates an event: `events/<timestamp>/frame.jpg` (1080p, JPEG q80) + `meta.json` (boxes, confidences, decision, mode, pan/tilt).
- Random negative frames: 1 per 10 min inside the active window.
- Disk cap 2 GB, oldest events deleted first.
- Dataset export: zip in YOLO format (images + label files from detections as pseudo-labels, to be reviewed before any training).

5.10 Deployment

- Built with Xcode, installed as a `.tipa` through TrollStore so it keeps its signature when the semi-tethered jailbreak is inactive. **The core system never depends on the jailbreak being active.**
- Phone settings: Auto-Lock never, brightness minimum, Low Power Mode off, Guided Access to pin the app.
- `idleTimerDisabled = true`; the camera requires the app in the foreground.
- Jailbreak extras, only when active: OpenSSH for debugging, a LaunchDaemon watchdog that relaunches the app if it is not running.

6. ESP32 firmware

PlatformIO, Arduino framework, NimBLE-Arduino. One environment per board type.

Module	Behaviour
BleLink	GATT server, one service, <code>cmd</code> (write) + <code>status</code> (notify), MTU 185

Module	Behaviour
Servos	LEDC 50 Hz, 500–2500 μ s, per-servo trim, slew limit 120°/s, hard angle limits from <code>cfg</code> (default pan –60..60°, tilt –30..40°); out-of-range commands are clamped. Park = (0, 0). <code>aim</code> and <code>park</code> are accepted armed or disarmed
Shooter	State machine IDLE → MOVE → SETTLE (150 ms) → OPEN (ms) → CLOSE → COOLDOWN (5 s). Hard cap 500 ms per burst. Rejects when disarmed, cooling down, tank empty, faulted or busy
Pump	Powered only while armed, tank ok and no fault. The pump's own pressure switch regulates pressure
Charger	iPhone USB 5 V switch. Follows <code>charge</code> commands. Defaults ON at boot and on link loss
Fan	ON if commanded or dry-zone temp > 40 °C, OFF below 35 °C
Safety	Heartbeat timeout 3 s → safe state: disarm, valve closed, pump off, park, charger ON. Dry-zone temp > 60 °C → fault OVERTEMP + disarm. Task watchdog 5 s
Sensors	Float switch (debounced 2 s) → <code>TANK_EMPTY</code> while low; DS18B20 every 5 s

The solenoid valve is normally closed, so any power loss leaves it shut.

7. Protocol (BLE, JSON)

- Service UUID: EC61AB6F-D20E-4217-93F9-4A3DF81B75D3
- `cmd` characteristic (write with response): 7C7FBA40-4383-4738-AD6B-09986229ED6A
- `status` characteristic (notify): 6DF54A5A-41DC-4414-AB6A-1354C959CF0A
- One JSON object per write/notify, $\leq$ 180 bytes. Angles in degrees relative to park.

Phone → ESP32:

```
{
  "c": "hb"
}
{
  "c": "arm", "v": true
}
{
  "c": "aim", "pan": 12.5, "tilt": -3.0
}
{
  "c": "park"
}
{
  "c": "shoot", "pan": 14.0, "tilt": -2.5, "ms": 300
}
{
  "c": "charge", "v": false
}
{
  "c": "fan", "v": true
}
{
  "c": "cfg", "panMin": -60, "panMax": 60, "tiltMin": -30, "tiltMax": 40
}
```

ESP32 → phone:

```
{
  "armed": true, "pan": 12.5, "tilt": -3.0, "tank": "ok", "pump": true, "charge": false, "fan": false, "temp": 31.2, "fault": null, "shots": 12
}
{
  "ack": "shoot", "ok": false, "why": "cooldown"
}
```

- Status is notified at 1 Hz and on every change.
- Every command except `hb` and `aim` gets an `ack`.
- `why` codes: `disarmed`, `cooldown`, `tank`, `fault`, `busy`, `bad`.
- Fault codes: `TANK_EMPTY`, `OVERTEMP`.
- `protocol/fixtures/*.json` holds canonical messages; both the Swift and firmware test suites parse and round-trip them.

8. Aim calibration

1. Switch to `calibration` mode in the web UI (pump on, detection off).
2. Jog the head with arrow buttons (1° / 5° steps) and press **Test shot** (150 ms).
3. The app buffers frames for 1.5 s after the shot and shows the frame with the largest difference from the pre-shot frame (the splash).
4. Click where the water landed and type the distance in metres (a tape measure or a paced estimate is fine). The point (x, y) → (pan, tilt, range_m) is saved.

5. Repeat for 10–20 points spread over the yard, covering 2–6 m. The UI shows the fit and per-point residuals; add points where residuals are large.
6. **Height offset**, at 3–4 points spread over the range, needed for head aim:
 - stand a 25 cm mark at the point (a bucket, or cardboard at cat height);
 - jog the tilt up until the water hits the mark;
 - press **Save height offset**, storing $\Delta = \text{tilt_mark} - \text{tilt_ground}$ against that point's range.
7. Draw the no-fire zone (area within ~2 m of the gnome) and ignore zones in the mask editor.

Calibration, masks and settings persist as JSON on the phone. Expected accuracy: ~1° servo play ≈ 10 cm at 6 m, against a ~40 cm cat body or a ~10 cm head. Every shot starts from full line pressure (pump pressure switch refills between shots), so range is repeatable.

9. Failure handling

Failure	Behaviour
BLE link lost	ESP32 enters safe state after 3 s. App reconnects continuously; web UI shows link state
App crash	Same as link lost. Jailbreak daemon relaunches app when jailbreak active; otherwise manual restart
Thermal serious / critical	See §5.7
Tank empty	Pump off, shots rejected with <code>tank</code> , web UI alert. Clears when float switch reads ok
Leak	Tank drains; float switch stops pump. Wet zone has drain holes
Camera session interrupted	Restart session; 3 failures within 5 min → disarm and alert
Power cut	Valve stays closed (NC). Phone runs on battery and disarms on link loss
iPhone reboot	App must be started manually (and jailbreak re-run if wanted). Web UI unreachable signals it
False positives	Only <code>cat</code> class, 2-of-3 confirmation, stillness required, dry-run week before live

10. Testing

- **DwarfCore:** `swift test` on macOS — MotionDetector on synthetic frames, Tracker association/confirmation/stillness, Aimer fit accuracy on synthetic calibration data, FirePolicy rule table (every shoot condition has a failing case), Protocol fixtures.
- **Replay harness:** macOS command-line target that runs recorded 1080p clips through the full pipeline including the CoreML model and asserts expected detections and decisions. Clips come from the gnome's own dataset capture.
- **Firmware:** `pio test -e native` — protocol parser against fixtures, Shooter state machine, Safety timeouts, cooldowns and caps with a fake clock. Manual hardware checks with the nRF Connect app.
- **Field:** one week in `dry-run`, review "would fire" events, tune thresholds, then switch to `live`.

11. Repository layout

```

dwarf/
  ios/DwarfCore/  Swift package: pure logic + tests + replay harness
  ios/DwarfApp/   Xcode app: camera, CoreML, BLE, web server, event store
  firmware/       PlatformIO project for ESP32
  protocol/       message schema + shared fixtures
  tools/          Python: YOLO → CoreML export, dataset utilities
  hardware/       BOM, wiring diagram, gnome layout notes
  docs/           specs and plans

```

12. Build order

Milestone	Content	Exit criteria
M0 Spikes	(a) YOLO11n CoreML on the 6s at 640; (b) BLE hello between app and ESP32	(a) ≥ 3 inferences/s sustained for 10 min without <code>serious</code> thermal state; if not met, try <code>imgsz 416/320</code> and re-plan the cycle budget. (b) heartbeat round-trips and survives ESP32 reboot
M1 Firmware + bench rig	All commands, safety state machine, valve spraying into a bucket	Native tests pass; manual nRF Connect checklist passes; measured jet reach ≥ 6 m
M2 Detection in dry-run	Pipeline, tracker, web UI, event store on the bench	Replay harness passes on recorded clips; live dry-run logs cats in the yard
M3 Calibration + live fire	Calibration UI, Aimer, FirePolicy live	Calibration residuals $\leq 2^\circ$; body shots hit a cat-sized target at 3 m; head shots hit a 25 cm mark at 5 m and 6 m
M4 Gnome integration	Wet/dry zones, window, vents, fan, wiring	1 sunny day in place without thermal <code>critical</code> ; no water in dry zone
M5 Field	One dry-run week, then live	Threshold review done; switched to <code>live</code>

13. Out of scope for v1

- Night operation.
- Telling the owner's own cats from intruders.
- Model fine-tuning (v1 only captures and exports data).
- Push notifications.
- Automatic restart after an iPhone reboot without a computer.
- Leading moving targets (v1 fires only at still cats).
- Posture-aware head height (v1 uses one 25 cm offset for any pose).
- Fully automatic splash-based calibration.
- Multiple gnomes.

14. Risks

Risk	Mitigation
YOLO too slow on A9 GPU	M0 benchmark first; smaller <code>imgsz</code> ; fewer crops per cycle
Cats missed at 6 m	~75 px per cat in 1080p; recall measured on replay clips in M2; 4 K capture is the fallback if recall is poor
iPhone overheating in gnome	Shade, light colour, vents, fan, thermal throttling; M4 measurement
Battery aging/swelling	40–80 % charge window; inspect monthly; battery is replaceable
12 V pump cannot reach 6 m	Measured in M1; adjust nozzle orifice; add accumulator if bursts are weak
Weak WiFi in yard	Affects only the web UI; core loop runs on BLE
Head shot lands on an eye or ear up close	No fire under 2 m; body aim 2–4 m; head aim only beyond 4 m, where the jet has spread